

The C-- Language Reference Manual

Simon Peyton Jones Thomas Nordin Dino Oliva
Pablo Nogueira Iglesias

May 23,1999

Contents

1	Introduction	3
2	Syntax definition	3
2.1	General	3
2.2	Comments	3
2.3	Names	6
2.4	Name scope	6
2.5	Reserved words	6
2.6	The <code>import</code> and <code>export</code> declarations	6
2.7	Constants	7
2.7.1	Integer and floating point numbers	7
2.7.2	Characters and strings	7
3	Fundamental concepts in C--	7
3.1	Memory	7
3.2	Data segment	8
3.3	Code segment	8
3.4	Types	8
3.5	Local variables (or registers)	8
3.6	Global variables	8
3.7	Addresses	9
3.8	Names	9
3.9	Foreign language interface	9
4	Data layout directives	9
4.1	Labels	10
4.2	Initialisation	10
4.3	Alignment	11
5	Procedures	12

5.1	Procedure definition	12
5.2	Local declarations	13
5.2.1	Local variables	13
5.2.2	Stack directive	13
5.2.3	Names usage	13
5.3	Statements	14
5.3.1	Null statement (;)	14
5.3.2	Assignment	14
5.3.3	Memory write	14
5.3.4	if and relational operations	15
5.3.5	switch	16
5.3.6	Local control labels and goto	16
5.3.7	Procedure call	17
5.3.8	jump	18
5.3.9	return	18
5.4	Foreign language interface	18
6	Expressions	19
6.1	Introduction	19
6.2	Memory read	20
6.3	Operators, precedence, and evaluation order	20
6.4	Primitives	21
6.5	Exception handling	22
6.6	Casting	23
6.6.1	bitsn	23
6.6.2	floatn	23
6.7	bits operators and primitives	23
6.7.1	+ and -	24
6.7.2	*	24
6.7.3	/	25
6.7.4	%	25
6.7.5	neg% and abs%	25
6.7.6	sign%	26
6.7.7	&, , and ^	26
6.7.8	~	26
6.7.9	<< and >>	26
6.8	float operators and primitives	26
6.8.1	+f, -f, and *f	27
6.8.2	/f	28

6.8.3	<code>signf%</code>	28
6.8.4	<code>negf%</code> and <code>absf%</code>	28
6.8.5	<code>exponentf%</code>	28
6.8.6	<code>fractionf%</code>	29
6.8.7	<code>scalef%</code>	29
6.8.8	<code>succf%</code> and <code>pred%f</code>	29
6.8.9	<code>ulpf%</code>	29
6.8.10	<code>truncf%</code>	30
6.8.11	<code>roundf%</code>	30
6.8.12	<code>intpartf%</code> and <code>fractpartf%</code>	30

7 Further Work 31

1 Introduction

C-- is a portable assembly language designed to be a good backend for high level languages (particularly for those that make use of garbage-collection) and to run fast on a number of todays major computer architectures. It is also designed to have as few dependencies as possible on the underlying hardware, but speed and ease of use has sometimes taken precedence over orthogonality and minimality. C-- should be rich enough to be a viable backend for most mainstream and research compilers.

This paper should be sufficiently self-supporting so that anyone who knows an imperative language and is acquainted with computers should be able to write her/his own C-- programs after reading this document.

2 Syntax definition

The syntax of C-- is given in Figures 1 and 2.

2.1 General

A C-- program file is written in eight bit ASCII characters. It consists in a sequence of `data` layout directives (Section 4), and/or procedure definitions (Section 5), and/or `import` declarations, and/or `export` declarations (Section 2.6), and/or `global` declarations (Sections 3.6), interleaved in any order.

A C-- *compilation unit* is a C-- program file that can be successfully compiled and that is suitable for linking.

C-- does not support input/output. Nevertheless, it can be accomplished using a foreign language call (Section 3.9).

2.2 Comments

Comments start with `/*` and end with `*/`. They can be nested.

<i>Program</i>	<i>program</i> → <i>pal</i> ₁ ... <i>pal</i> _{<i>n</i>}	<i>n</i> ≥ 0
<i>Pal</i>	<i>pal</i> → <i>data</i> [<i>conv</i>] <i>Name</i> (<i>arg</i> ₁ , ... <i>arg</i> _{<i>n</i>}) [<i>data</i>] <i>body</i> import <i>Name</i> ₁ , ... <i>Name</i> _{<i>n</i>} ; export <i>Name</i> ₁ , ... <i>Name</i> _{<i>n</i>} ; global { <i>global</i> ₁ ... <i>global</i> _{<i>n</i>} }	<i>n</i> ≥ 0 <i>n</i> ≥ 1 <i>n</i> ≥ 1 <i>n</i> ≥ 1
<i>Data</i>	<i>data</i> → data { <i>datum</i> ₁ ... <i>datum</i> _{<i>n</i>} }	<i>n</i> ≥ 1
<i>Datum</i>	<i>datum</i> → <i>Name</i> : <i>type</i> [<i>cexpr</i>][{ <i>cexpr</i> ₁ , ... <i>cexpr</i> _{<i>n</i>} }] ; <i>type</i> [] { <i>cexpr</i> ₁ , ... <i>cexpr</i> _{<i>n</i>} } ; bits8 [] <i>AsciiString</i> ; bits16 [] <i>UnicodeString</i> ; align <i>n</i> ;	<i>n</i> ≥ 1 <i>n</i> ≥ 1 <i>Abbreviation</i> <i>Abbreviation</i> <i>Alignment directive</i>
<i>Constant Expressions</i>	<i>cexpr</i> → <i>numexpr</i> <i>const</i>	
<i>Simple Constants</i>	<i>sconst</i> → <i>Int</i> 'char' unicode ('char')	<i>Integer constant</i> <i>Ascii char. constant</i> <i>Unicode char. constant</i>
<i>Constants</i>	<i>const</i> → <i>sconst</i> <i>Name</i> <i>AsciiString</i> <i>UnicodeString</i>	<i>Simple constants</i> <i>Symbolic constant</i> <i>String constant</i> <i>Unicode string constant</i>
<i>Numeric Operators</i>	<i>numOp</i> → + - * /	
<i>Numeric Constants</i>	<i>num</i> → <i>Int</i> <i>Float</i>	
<i>Numeric Expressions</i>	<i>numexpr</i> → <i>num</i> <i>Name</i> (<i>cnumexpr</i>) <i>cnumexpr</i> <i>numOp</i> <i>cnumexpr</i>	
<i>Strings</i>	<i>AsciiString</i> → "char ₁ ... char _{<i>n</i>} " <i>UnicodeString</i> → unicode ("char ₁ ... char _{<i>n</i>} ")	<i>n</i> ≥ 0 <i>n</i> ≥ 0
<i>Convention</i>	<i>conv</i> → foreign <i>convkind</i>	<i>Convention declaration</i>
<i>Conventions</i>	<i>convkind</i> → C Pascal ...	<i>Calling Conventions</i>
<i>Formal Arguments</i>	<i>arg</i> → <i>type Name</i>	
<i>Type</i>	<i>type</i> → bits <i>n</i> float <i>n</i>	<i>n</i> ∈ {8, 16, 32, 64} <i>n</i> ∈ {32, 64}
<i>Function Body</i>	<i>body</i> → { <i>decl</i> ₁ ... <i>decl</i> _{<i>n</i>} <i>stm</i> ₁ ... <i>stm</i> _{<i>m</i>} }	<i>n</i> ≥ 0, <i>m</i> ≥ 1
<i>Local Declarations</i>	<i>decl</i> → <i>type Name</i> ₁ , ... <i>Name</i> _{<i>n</i>} ; stack { <i>stackdatum</i> ₁ ... <i>stackdatum</i> _{<i>n</i>} }	<i>Var. decl.</i> , <i>n</i> ≥ 1 <i>n</i> ≥ 1

Figure 1: C-- syntax

<i>Statements</i>	<i>stm</i>	→ ; <i>Name</i> = <i>expr</i> ; <i>type</i> [<i>{alignn}</i>][<i>expr</i>] = <i>expr</i> ; if <i>expr</i> <i>rel</i> <i>expr</i> <i>block</i> [else <i>block</i>] switch[<i>[sconst</i> ₁ ... <i>sconst</i> _{<i>n</i>}]] <i>expr</i> { <i>swt</i> ₁ ... <i>swt</i> _{<i>n</i>} } <i>Name</i> : goto <i>Name</i> ; jump <i>expr</i> (<i>expr</i> ₁ ,... <i>expr</i> _{<i>n</i>}) ; [<i>conv</i>] [<i>Name</i> ₁ ,... <i>Name</i> _{<i>m</i>} =] <i>expr</i> (<i>expr</i> ₁ ,... <i>expr</i> _{<i>n</i>}) ; [<i>conv</i>] return(<i>expr</i> ₁ ,... <i>expr</i> _{<i>n</i>}) ;	<i>Null statement</i> <i>Assignment</i> <i>Memory write, align. n</i> <i>n</i> ≥ 1 <i>Local control label</i> <i>Goto local label</i> <i>n</i> ≥ 0, <i>Jump to expr</i> <i>n, m</i> ≥ 0 <i>n</i> ≥ 0
<i>Block</i>	<i>block</i>	→ { <i>stm</i> ₁ ... <i>stm</i> _{<i>n</i>} }	<i>n</i> ≥ 0
<i>Switch branch</i>	<i>swt</i>	→ <i>sconst</i> ₁ ,... <i>sconst</i> _{<i>n</i>} : <i>block</i> default : <i>block</i>	<i>n</i> ≥ 1
<i>Expressions</i>	<i>expr</i>	→ <i>const</i> <i>Name</i> <i>type</i> [<i>{alignn}</i>][<i>expr</i>] (<i>expr</i>) <i>expr</i> <i>op</i> <i>expr</i> <i>prim</i> (<i>expr</i> ₁ ,... , <i>expr</i> _{<i>n</i>})	<i>Variable or label</i> <i>Memory read, align. n</i> <i>n</i> ≥ 1
<i>Operators</i>	<i>op</i>	→ + <i>flag</i> - <i>flag</i> * <i>flag</i> / <i>flag</i> % <i>flag</i> & ^ << >> <i>flag</i> ~	<i>Arithmetic</i> <i>Bitwise</i>
<i>Primitives</i>	<i>prim</i>	→ neg% <i>flag</i> negf% abs% <i>flag</i> absf% sign% signf% exponentf% <i>flag</i> fractionf% <i>flag</i> scalef% <i>flag</i> succf% <i>flag</i> prefd% <i>flag</i> ulpf% <i>flag</i> truncf% roundf% <i>flag</i> intpart% fractpart% <i>type flag</i>	 <i>Type Casts</i>
<i>Flags</i>	<i>flag</i>	→ o u t ut f fz fn fp ft ftz ftn ftp	<i>No Flag</i> <i>UnOrdered</i> <i>Unsigned and Trapping</i> <i>Floating and Rounding</i> <i>Floating and Trapping</i>
	<i>compflag</i>	→ u o fo	<i>No Flag</i>
<i>Relations</i>	<i>rel</i>	→ == <i>compflag</i> != <i>compflag</i> > <i>compflag</i> < <i>compflag</i> >= <i>compflag</i> <= <i>compflag</i>	
<i>Global Declarations</i>	<i>global</i>	→ <i>type Name</i> ; <i>type Name</i> register ;	
<i>Stack Datum</i>	<i>stackdatum</i>	→ <i>Name</i> : <i>type</i> [<i>[cexpr]</i>] ; alignn ;	<i>Alignment directive</i>

Figure 2: Statements in C--

2.3 Names

Names are made up of letters, digits, underscore and dots. A name cannot begin with a number character or with a dot followed by a number character. Upper and lower case are distinct. Imported names should also follow these restrictions.

Names are identifiers for registers or memory addresses (Section 3.8).

The following are examples of legal C-- names:

```
x
foo
_912
aname12
_foo.name_abit_12.long
Sys.Indicators
```

These are two illegal C-- names:

```
.9Aname
3illegal
```

2.4 Name scope

Procedure and label names are always global inside a C-- compilation unit (or program). Local variable names, formal argument names, stack label names and local control labels are only in scope of the procedure body where they are declared. There is no nested scoping of names inside a procedure. A local name shadows a global name (C-- compilers may choose to emit a warning in these cases). Procedure and label names may be used before they are declared.

2.5 Reserved words

The following are reserved words and cannot be used as names for registers or memory addresses:

```
abs% absf% align C data default else eexponentf% xport float32
float64 foreign fractionf% fractpartf% global goto if intpartf% import
jump neg% negf% predf% register return roundf% scalef% sign% signf%
stack succf% switch truncf% ulpf% bits8 bits16 bits32 bits64
```

2.6 The import and export declarations

Names that are to be used outside of the C-- program must be exported with the **export** declaration. Likewise, names that the C-- program uses and does not declare must be imported with the **import** declaration. Only procedure and (pointer) label names may be exported.

Imported names should follow the syntactic restriction mentioned in Section 2.3.

An example where a few C external names are imported and a few C-- names are exported:

```
import printf, sqrt; /* C procedures used in this C-- program */
export foo, bar;     /* To be used outside this C-- program */
```

Names that are explicitly exported and imported are guaranteed to be unchanged by the compiler. All other names might be renamed.

The type of a name listed in an **import** declaration is the native pointer type of the architecture.

An **import** or an **export** declaration may appear anywhere in the program where a **data** layout directive or a procedure definition does.

2.7 Constants

Constants can be (signed) integers, (signed) floating point numbers, characters, strings and names. C++ follows C's syntax for denoting integer, floating point, character, and string constants.

2.7.1 Integer and floating point numbers

Integer constants have of type `bits`. Floating point constants have type `float`. Their size is architecture-dependent.

2.7.2 Characters and strings

Character and string constants are treated as integers and as pointer labels respectively. Character constants are ASCII characters surrounded by single quotes. String constants are a sequence of ASCII characters surrounded by double quotes.

A character constant is treated as an integer whose value is the character's 8-bit ASCII code. Therefore, character constants have type `bits8`. C++ uses C's escape sequences to denote special characters, such as `\n` for the new line and `\t` for the tabulator.

For example, character constant `'H'` is a `bits8` with value 72.

String constants are like labels that point to the first `bits8` of an array of `bits8`s stored in static memory. Therefore, they have type `bitsn` where n is the particular architecture's natural pointer size. *String constants are not automatically null-terminated.*

For example, the string `"Hello World"` is viewed as a label that points to the first byte of the array of bytes with values 72, 101, 108, 108, 111, 32, 87, 111, 114, 108, 100, stored in static memory.

It is possible to have UTF-8 integers for single characters and for string characters.¹

The syntax to specify an UTF-8 constant is:

```
unicode(constant)
```

where *constant* is a character constant or a string constant. The type of UTF-8 characters is `bits16`, for it requires two bytes—two ASCII characters—to code a Unicode character. UTF-8 strings are pointers to the first `bits16` of an array of `bits16`s stored in static memory, and therefore, they have type `bitsn`, where n is the architecture's natural pointer size.

3 Fundamental concepts in C++

3.1 Memory

Memory is an array of bytes from which different sized *types* (Section 3.4) can be read and written. The size of the addressable memory is implementation dependent (Section 3.7). All addresses and offsets are specified in bytes. No guarantee about endianness is given, i.e. a portable program should either not depend on a specific endianness or find it out.

¹UTF-8 is an encoding of Unicode characters into 8-bit ASCII characters that does not use any of the ASCII control characters to perform the coding. Unicode is an abbreviation for Universal Multiple-Octet Coded Character Set (UCS), and it is defined in ISO/IEC 10646. It is an international standard for encoding computer character sets that differs from historical ASCII. UTF-8 stands for Universal Transformation Format, 8-Bit form. See <http://www.unicode.org/unicode/standard/utf8.html> for more information.

3.2 Data segment

The data segment is the part of memory where the static (initialised or uninitialised) data is allocated. The data segment is read/write, so the values stored can be changed at runtime. The size and initial content of the data segment is determined at compile time (Section 4). C++ does not provide dynamic memory allocation natively, nonetheless, it can be accomplished with foreign language calls (Section 3.9 and Section 5.4).

3.3 Code segment

The code segment is the part of memory where the executable program code is stored (Section 5). The code segment consists of a series of procedure definitions.

C++ does not currently provide a mechanism for creating code at runtime.

3.4 Types

There are only two kinds of types provided by C++, namely `bits` and `float`. These types can have different sizes.

- The size of a `bits` can be 8, 16, 32 or 64 bits.
- The size of a `float` can be 32 or 64 bits.

A type must be qualified with a size. Thus `bits8` and `bits16` are different types.

There is no pointer type. The `bits $n$` type can be used for pointers (addresses), where n is the particular architecture's *natural pointer size*, i.e.: n is the number of bytes needed to hold a memory address in the particular architecture.

For example, a four byte word is specified as `bits32`, an eight byte float is specified as `float64` and so on.

Types are used in

1. **Local variable declarations** (Section 5.2.1), to declare the type of local variables.
2. **Memory write statements and memory read expressions** (Section 5.3.3 and Section 6.2, respectively) to indicate the type of the value written/read.
3. **Data layout directives** (Section 4) to indicate the type of the allocated datum.

3.5 Local variables (or registers)

Any number of local variable names may be declared inside procedure bodies. Local declarations must appear before the statements in the text of the procedure body. They are typed storage locations that don't have an address. The term "local variable name" is interchangeable with the term "register", since there is an unlimited supply of (virtual) registers: i.e. a local variable name will be mapped to a machine register if there is one available, otherwise it is mapped to a memory location (e.g. the stack), but the mapping is transparent and local variables should be viewed as registers.

3.6 Global variables

Global variables are like local variables except that the scope of a global is *all* the functions of a C++ compilation unit.

Globals differ from data declarations in that `data` allocates memory, and a data label refers to the address of a datum. Globals, like locals, should be viewed as registers and don't have addresses.

For example, given the declaration

```
global {
    bits64 hp;
}
```

the implementation attempts to put variable `hp` in a register but, like locals and procedure parameters, it may put it in memory. `global` declarations may name specific registers, like

```
global {
    bits64 hplim "%ebx";
}
```

Register names are implementation-dependent.

All separately compiled modules must have *identical* `global` declarations.

3.7 Addresses

To specify where in memory to read or write we need an address. Any expression that evaluates to a `bitsn` can be used as an address, where n is the architecture's *natural pointer size*, i.e.: n is the number of bytes needed to hold a memory address in the particular architecture.

Absolute addresses can be used but what they refer to is implementation dependent. Their type is also `bitsn`.

3.8 Names

A name declares either a register or a memory address. Register names are procedure's local variables. Memory address names are either labels (Section 4.1) or procedure names (Section 5).

3.9 Foreign language interface

The foreign language interface is a way for C-- programs to use other calling conventions for procedure inter-operation with foreign code. This interface is nearly 100% portable across architectures (Section 5.4).

4 Data layout directives

Memory in the data segment is allocated using the `data` directive. A memory block is organised as a sequence of typed data. Each *datum* is thought of as an array of bytes that may be initialised.

Here is an example that allocates and initializes some memory. In particular, it allocates 8 datums of types `bits32`, `bits16`, `float64`, `bits32`, `bits8`, `bits16`, `bits8` and `bits64` respectively. The example is explained in more detail in the remainder of this section:

```
data {
    foo: bits32[4]{1,2,3,ff}; /* ff is a forward reference */
        bits16[4]{1,2};
    ff:  float64[2]{2.8,3.1}; bits32[2]{ff,foo};
    str: bits8[]"Hello world\0";
    ustr: bits16[]unicode("Hello world\0");
        bits8;
```

```

    xs:    bits64[]{"This is an", "array of", "bits64's"};
}

```

There may be any number of data layout directives in a C++ program.

4.1 Labels

Labels are the means to refer to the allocated memory. They should be viewed as pointers and not as memory locations. A label declaration consists of a name followed by a colon. Once declared, a label is a name (and so an expression) that refers (points) to a memory address. Therefore it has `bits $n$` type, where n is the particular architecture's natural pointer size. Labels may be used before their are declared, e.g. label `ff` is used in the initialisation of the data directive's first datum before it is declared pointing to the third.

Note that labels do not provide any information about the type of the data pointed to by them.

A label points to the first byte after its declaration. Here is an example in which four labels point to the same datum:

```

data { foo: label1: label2: bar: bits32} /* just allocates */

```

Memory is always allocated without padding inside a single data layout directive, so it is possible to find any given data in the data segment by starting from a label and adding the right offset, as in, for example, the read expression `bits16[foo+4]`. Indeed, `foo+4` does not have to point to the beginning of a data element. It may point to any other data byte, but it is assumed by C++ that it is 2-byte aligned.

To align a label (and hence the datum it points to) to a specific boundary, an alignment directive (Section 4.3) has to be placed before the label. In the following example, `foo` and `bar` might or might not be the same address, but `bar` is guaranteed to be aligned on an eight byte boundary.

```

data { foo: align8;
      bar: bits32{0};
}

```

It is possible to have a stupid data layout directive with no labels that is inaccessible.

4.2 Initialisation

Memory is allocated by specifying the type of the datum, the number of datum's elements to allocate, and the initial value for each element. The particular syntax is:

```

type[n]{constant-list};

```

where n specifies how many elements of the type `type` have to be allocated, and `constant-list` provides the initial value (of type `type`) for each allocated element, in the form of a comma-separated list of constants or constant expressions (i.e. expressions whose value is known at compile time).

There are a number of possible variants:

1. If `[n]` is not provided, only one element is allocated. The `{constant-list}` may or may not be provided. If provided, it should contain only one constant or constant expression to which the element is initialised. If not provided, no initial value is given. For example:

```

data { lb1: bits8; }
/* Allocates one byte (contains garbage) */
data { lb2: bits8{17}; }
/* Allocates one byte and initialises it to (ASCII code) 17 */
data { lb3: bits32{17}; }
/* Allocates one 4-byte word and initialises it to integer 17 */

```

2. If $[n]$ is provided, then n elements are allocated. The $\{constant-list\}$ may or may not be provided. If not provided, no initial value is given. If provided, it should contain c constants or constant expressions, such that $c \leq n$. Element i ($i : 0 \dots (n - 1)$) is initialised to the value of the constant or constant expression j ($j : 0 \dots (c - 1)$) in $\{constant-list\}$, such that $j = i \bmod c$. For example:

```
data { lb1: bits8[17]; }
/* Allocates 17 bytes (that contain garbage) */
data { lb2: bits8[17]{0}; }
/* Allocates 17 bytes and initialises all of them to 0 */
data { lb3: bits32[6]{1,2,3}; }
/* Allocates six 4-byte words and initialises them
 * to 1,2,3,1,2,3 respectively.
 */
data { lb4: bits32[6]{1,2,3,1,2,3}; }
/* Allocates six 4-byte words and initialises them
 * to 1,2,3,1,2,3 respectively.
 */
data { lb5: bits32[4]{1,2,3}; }
/* Allocates four 4-byte words and initialises them
 * to 1,2,3,1 respectively.
 */
```

3. There is also the possibility to have abbreviations when $n = c$.

```
type[]{constant-list};           is an abbreviation for  type[c]{constant-list};
bits8[]"char1...charn";         is an abbreviation for  bits8[n]{'char1',..., 'charn'};
bits16[]unicode("char1...charn"); is an abbreviation for  bits16[n]{unicode('char1'),..., unicode('charn')};
```

For example:

```
data { s1: bits8[6]{'h','e','l','l','o','\0'}; }
data { s2: bits8[]"hello\0"; }
/* Both directives allocate 7 bytes and initialise
 * them to the same ASCII code integers.
 */
data { f1: float64[3]{3.5, 4.4, 6.98}; }
data { f1: float64[] {3.5, 4.4, 6.98}; }
/* Both directives allocate three 8-byte floats and initialise
 * them to the same floating point numbers.
 */
```

Since the initialised value might have dependencies on the endianness, the only way to guarantee that a memory read (Section 6.2) gets the same initialised (or written) value, is to read the datum or the element with the same type as it was initialised (or written). For example, if a datum was initialised with `data {foo: bits16{17};}`, if read back with `bits8[foo]` the value might be 0 or 17 depending on the architecture, but if read with `bits16[foo]` it is guaranteed to be 17.

4.3 Alignment

For performance reasons, and also to comply with some architecture requirements, it is sometimes necessary to specify the alignment of data. The `alignn` directive inserts padding as needed, ensuring that the next datum is placed on an n byte boundary. The value of the padding is unspecified. The alignment value n must be a power of 2 (2,4,8,16 ...).

In the following example, `foo` is aligned to a 4-byte boundary and `bar` to an 8-byte boundary. In both cases padding may be inserted: for example, between the last byte of the `bits32` and the first byte of the `float64` (the datum pointed to by `bar`) there may be padding in order to place the `float64` on an 8-byte boundary.

```

data { align4;
      foo: bits32{1};
      align8;
      bar: float64{1.7};
    }

```

In a sequence of several `align $n$` directives, the one with the higher alignment value is considered. `align $n$` directives appearing at the end of a `data` declaration are ignored. In the following example, the second `bits8` is placed on an 8 byte boundary and the last `align4` is ignored.

```

data { foo: bits8;
      align4;
      align8;
      align2;
      bits8;
      align4;
    }

```

5 Procedures

Procedures are the means to place information in the code segment. They are very similar to high-level language procedures. Procedures can optionally take arguments, contain static data declarations and return values.

5.1 Procedure definition

A procedure definition has the following syntax:

$$[conv] \textit{proc_name}(\textit{type } arg_1, \dots, \textit{type } arg_n) \textit{ [data] } \{ \textit{body} \}$$

where:

- *conv* is the (optional) calling convention declaration (Section 5.4) needed for inter-operation.
- *proc_name* is the procedure *name*, which stands for the procedure entry point address and, as a name, will be an expression (Section 3.8). It may be also used in call (Section 5.3.7) and `jump` (Section 5.3.8) statements.
- *type arg₁, ..., type arg_n* is the formal argument list. Each formal argument consists of a *name* preceded by its type. If the procedure takes no arguments, the list should be empty. Unlike C, procedures with a variable number of arguments are not supported. The formal arguments are only in scope of the procedure body.
- *data* is the (optional) data declaration.
- *body* is the procedure body enclosed in curly braces. It consists of a (possibly empty) sequence of local declarations (variable declarations and stack directives), followed by a (non-empty) sequence of statements, such as assignments, memory writes, local control label declarations, `gotos`, etc. It is unspecified what happens if control flows off the edge of a procedure body, that is, every control path in a procedure body should finish in a `jump` (Section 5.3.8) or in a `return` (Section 5.3.9) statement. That is why a procedure body should have at least one (non null) statement.

The return type needs not be specified in the definition.

For example, procedure `foo` is defined as a procedure that expects one `bits32` argument. Inside the procedure body, the local variable (or register) `x` is declared, followed by an assignment statement and a `jump` statement to procedure `bar`.

```
foo(bits32 y) {
    bits32 x;

    x = y + 1;
    jump bar(x);
}
```

5.2 Local declarations

5.2.1 Local variables

A local variable declaration has the following syntax:

```
type name1, ..., namen ;
```

It declares the local variable names $name_1 \dots name_n$ of type *type*. These names will be mapped to (virtual) machine registers. As names, they are also expressions of type *type*.

Local variables have to be declared before they are used.

All declarations must appear at the beginning of the procedure body. All the local variable names must be unique. It is not possible to redeclare a name.

5.2.2 Stack directive

To handle high-level variables that can't be represented using C--'s primitive types, C-- can be asked to allocate named areas in the procedure's activation record.

```
f (bits32 x) {
    bits32 y;

    stack { p : bits32;
            q : bits32[40];
    }
    /* Here, p and q are the addresses of the relevant chunks of
       data. Their type is the native pointer type of the machine. */
}
```

`stack` is rather like `data`; it has (almost) the same syntax between the braces, but it allocates on the stack. The only difference is that in stack allocated memory cannot be initialised. As with `data`, the names are bound to the (stack) addresses of the relevant locations.

A word of caution here: in the example above, `p` is bound to an address in `f`'s activation frame. *It only makes sense to read from/write to address `p` while `f`'s frame is still on the stack, i.e., before `f` returns or jumps to another function*. After `f` has returned, address `p` will refer to some other function's activation frame or may even be beyond the current stack pointer!

C-- makes no provision for dynamically-sized stack allocation (yet).

5.2.3 Names usage

If a local name (formal argument, local variable or stack label) is the same name as a label (Section 4.1) or a procedure name, the uses of the name within a procedure refer to the local. That is, the local

name shadows the global name (Section 2.4).

5.3 Statements

5.3.1 Null statement (;)

This is just the null statement and can be inserted anywhere an ordinary statement can. It does not have any effects.

5.3.2 Assignment

An assignment statement has the following syntax:

```
name = expr ;
```

It stores the value of *expr* in the local variable (or register) *name*, where *expr* has the same type as *name*.

5.3.3 Memory write

A memory write statement has the following syntax:

```
bitsn[expr1] = expr2 ;
```

to write **bitsn** values, or

```
floatn[expr1] = expr2 ;
```

to write **floatn** values.

Expression *expr*₁ has type **bitsn**, where *n* is the particular architecture's natural pointer size, and its value is the memory address in which the value of *expr*₂ is written. Expression *expr*₁ will typically contain one or more labels. Expression *expr*₂ should be of type **bitsn** or **floatn** respectively, otherwise the value written in memory is unspecified.

The following example stores the ASCII integer code of 'A' in the 4th byte of the datum pointed to by **label**

```
bits8[label+4] = 'A';
```

The address yielded by *expr*₁ is assumed aligned to the size of the type, namely, *n*. A memory write can optionally be qualified with an alignment flag **{aligna}**, so the syntax is now:

```
bitsn{aligna}[expr1] = expr2 ;  
floatn{aligna}[expr1] = expr2 ;
```

A few examples of memory writes with flagged alignment:

- **float64**{**align4**}[**label**] = *expr* does a 8-byte write but assumes that **label** is aligned to a 4 byte boundary.
- **bits32**{**align1**}[**label**] = *expr* does a 4-byte write but assumes that **label** is aligned to a byte boundary (pointer to a byte).
- **bits8**{**align4**}[**label**] = *expr* does a 1-byte write but assumes that **label** is aligned to a 4 byte boundary.

5.3.4 if and relational operations

Conditional execution of code is accomplished with the `if` statement. It has the following syntax:

```
if expr1 rel expr2 { ... } [ else { ... } ]
```

The `else` branch is optional and the statement blocks may be empty, as in `if x == 0 {}`, but the curly braces are mandatory even for single statements, as in

```
if x == 0 { x = x + 1; }
```

The condition test is very simple: it consists of a relational operation, *rel*, that takes two expressions as arguments. The term “operation” is used instead of “operator”, therefore avoiding confusion with C++ operators that are used in expressions (Section 6). Relational operations are only used in `if` condition tests; they cannot be used anywhere else.

This is the set of relational operations:

<i>Name</i>	<i>Relation</i>
<code>==</code>	<i>Equality</i>
<code>!=</code>	<i>NonEquality</i>
<code>></code>	<i>Greater Than</i>
<code>>=</code>	<i>Greater Than or Equal</i>
<code><</code>	<i>Less Than</i>
<code><=</code>	<i>Less Than or Equal</i>

They can all be combined with these flags:

<i>Flag</i>	<i>Meaning</i>
	<i>Signed comparison (default)</i>
<code>u</code>	<i>Unsigned comparison</i>
<code>f</code>	<i>Floating point comparison</i>
<code>fo</code>	<i>Floating point unordered comparison, if supported</i>

When the condition test holds, the block of statements immediately following the condition test is executed. Otherwise, if an optional `else` branch has been specified, its block of statements is executed. After execution of the any of these blocks, control resumes at the first statement after the `if` or `if/else`.

In the following example, `>=` is used in the `if` test condition without a flag (default signed comparison), and `!=` is used combined with flag `u` to test whether the unsigned integer held in `x` is zero.

```
f(bits32 x)
{
    bits32 y;

    y = 0;
    if y >= bits32[foo+8] {
        y = y + 1;
        return (y);
    } else {
        x = x -u 1;
        if x !=u 0 {
            y = y + 2;
        }
        return (y);
    }
}
```

5.3.5 switch

The **switch** statement performs multiway branching depending on the value of a **bits** expression. The particular syntax is:

```
switch [sconst1..sconstn] expr {  
    sconst11, ..., sconst1i : {...}  
    ⋮  
    sconstm1, ..., sconstmj : {...}  
    default : {...}  
}
```

where:

- *expr* is an expression that yields a **bits**_{*n*} value.
- *sconst*_{*k*1}, ..., *sconst*_{*k**l*}: {...} is branch *k* (*k* : 1...*m*), in which multiple simple-constant² alternatives (*l* : 1...*i*, *j*...) may be specified.
When the value of *expr* is any of *sconst*_{*k*1}...*sconst*_{*k**l*}, branch *k* is taken, executing its block of statements and resuming control at the first statement after the **switch**. There is no fall through between different branches: C-- assumes that earlier branches are more likely to be taken.
- **default** is the (optional) default branch that is taken when none of the others are taken. The effect is unspecified if none of the branches are taken—none of the *sconst*_{*ij*} match *expr*'s value—and no default branch is provided.
- [*sconst*₁..*sconst*_{*n*}] is an (optional) range of simple constants in which the value of *expr* is guaranteed to be. This range is a hint to the compiler. No bounds checking is performed at run-time to see whether *expr*'s value is in the range.

In the following example, expression **x+23** is assumed to yield a value in between 0 and 7. If the value is 1,2 or 3, then the first branch is taken. If the value is 5, then the second branch is taken. If the value is 0,4,6, or 7, then the **default** branch is taken.

```
switch [0..7] x + 23 {  
    1,2,3    : { y = y + 1; }  
    5        : { y = x + 1; x = y; }  
    default : { y = f();  
                if y == 0 { x = 1; }  
            }  
}
```

5.3.6 Local control labels and goto

Local control labels are used in conjunction with the **goto** statement to alter the control flow within a procedure body. A local control label declaration consists of a label name followed by a colon. This kind of control label is not a *name* in the sense of Section 3.8, and so, it should not be confused with the pointer labels mentioned so far. The *only thing* that can be done with a local control label is to provide it as argument to **goto** statements.

In turn, a **goto** statement transfers control to the label it takes as argument. Only a local control label can be the argument of a **goto**.

In the following example, the **goto** statement forces the control flow to resume to the very first statement after the label declaration.

²That is, **bits** integers or characters. See Figure 1.


```

bar()
{
label:
    bits64[foo] = 18;
    bits64[foo+4*8] = bits64[bar];
    goto label;
    return();
}

```

5.3.7 Procedure call

A call statement invokes a procedure in the conventional way of function invocation, so all the invoking procedure's local variables are saved across the call. The particular syntax is:

$$[\textit{conv}] [\textit{name}_1, \dots, \textit{name}_n =] \textit{expr} ([\textit{expr}_1, \dots, \textit{expr}_m]);$$

where:

- $\textit{name}_1, \dots, \textit{name}_n =$ is the local variable name list. The results returned back by the procedure are stored in each variable in the order in which they are returned, from left to right, by the invoked procedure's **return** statement (Section 5.3.9). If the invoked procedure returns no values, the name list should be omitted, otherwise the values of the names are unspecified after the call.
- $\textit{conv}$ is the (optional) calling convention declaration needed for inter-operating with foreign code. (Section 5.4)
- $\textit{expr}$ is any expression that evaluates to a procedure address. It will typically be a (procedure) name.
- $\textit{expr}_1, \dots, \textit{expr}_m$ is the (optional) actual argument list, where each actual argument $\textit{expr}_i$ is an expression. All the expressions are passed by value to the called procedure. If no arguments are passed, the list should be empty, as in, for example, $\textit{x} = \textit{f}();$.

It is unspecified what the effects are if the number and the types of the actual arguments in a call statement do not match the number and the types of the formal arguments of the invoked procedure. It is also unspecified what the effects are if the number and the types of the names in the name list do not match the number and the types of the results returned by the invoked procedure.

The order of evaluation of the arguments matters, because the evaluation of an expression can raise an exception. A C++ compiler may evaluate arguments in any order.

Call statements are not expressions and so cannot be used inside expressions. Procedure calls are complete statements. Things such as $\textit{y} = \textit{f}(\textit{g}(\textit{x})) + 1;$ are not allowed. Recall, however, that procedure *names*, as such, are expressions with the procedure address as value.

The following example is self-explanatory:

```

foo()
{
    bits32 x, y;
    x, y = bar(5);
    return (x,y);
}
bar(bits32 x)
{
    return (x, x+1);
}

```

5.3.8 jump

The **jump** statement performs a control jump but carrying parameters. It has as target any expression that evaluates to a procedure address and can optionally transfer arguments to that procedure. The syntax is:

```
jump expr(expr1, ..., exprn);
```

where:

- *expr* is any expression that evaluates to a procedure address. It will typically be a (procedure) name.
- *expr*₁, ..., *expr*_{*n*} is the (optional) actual argument list, where each actual argument *expr*_{*i*} is an expression. All the expressions are passed by value to the target procedure. If no arguments are passed, the list should be empty, as in, for example, **jump bar ()**;

All local variables die when jumping. It is unspecified what the effects are if the number and the types of the actual arguments in a **jump** statement do not match the number and the types of the formal arguments of the invoked procedure. An example of an infinite loop with no stack growth:

```
bar(bits32 x, bits32 y)
{
    jump bar(y, x);    /* Loop forever */
}
```

5.3.9 return

The **return** statement transfers control back to the call statement issued by an invoking procedure. Optionally, it can also transfer values back. All the local variables of the procedure issuing the **return** die when returning. The syntax is:

```
[ conv ] return (expr1, ..., exprn);
```

where *expr*_{*i*} are the expressions whose values will be returned. If no values are returned, the expression list should be empty, as in **return ()**; . Note that in C--, a procedure may return multiple values.

The return statement may be qualified with the calling convention to be used (Section 5.4)

It is unspecified what the effects are if the number and the types of the values returned do not match between a **return** and the call statement.

```
bar(bits32 z)
{
    return (1+z, z/3);
}
foo(bits32 z)
{
    return ();
}
```

5.4 Foreign language interface

To use a foreign language calling convention for a procedure, the name of the calling convention should be declared before the procedure name with the **foreign** keyword. Here, **foo** uses the standard C calling convention.

```

export foo;
foreign C foo()
{
    bits32 x;
    jump bar(x);
}

```

The calling convention should be also specified in the same way in call statements and in **return** statements, if it is not C--'s calling convention.

```

import printf, fun;
goo()
{
    bits32 i;
    foreign C fun(5);
    /* fun has type int -> void */
    foreign C i = printf(str, arg);
    /* printf() returns an int */
    return ();
}
bar(bits32 a)
{
    a = a + 1;
    foreign C return (a); /* uses C's convention to return 'a' */
}

```

There supported calling conventions are:

1. C
2. Pascal

All foreign language functions/procedures must have been imported with import declarations. All C-- procedures directly invoked from a foreign language must have been exported with export declarations.

When calling a C-- procedure from a foreign program, the types and sizes of the actual arguments should match the types and sizes of the formal arguments in the particular platform, otherwise the effects are unspecified. The same applies for the types and sizes of returned values.

Since the size of a particular foreign language type may differ between platforms, and since C-- types are fixed-size types, it is impossible for C-- to be completely platform independent when inter-operating with foreign languages.

6 Expressions

6.1 Introduction

An C-- expression can be a constant, a name, a memory read, a primitive, or an operator applied to other expressions. C-- makes a distinction between integer and floating point expressions, i.e., expressions that yield **bitss** or **floats** as result.

The integer and floating point model is based on the LIA-1 standard (ISO/IEC 10967-1:1994(E)) and if there are any inconsistencies between this manual and LIA-1, the LIA-1 standard is correct, unless otherwise noted.

Signed and unsigned numbers are not distinguished. Instead, like any other assembler, it is the operations that are typed.

The type of any subexpression is always known and there are no automatic type casts or type conversions.

The following sections cover all the C++ operators, all the C++ primitives, and the memory read expression.

6.2 Memory read

Memory read expressions have the following syntax:

```
bitsm[expr]
Type: bitsn → bitsm
```

to read a `bitsm` value, and

```
floatm[expr]
Type: bitsn → floatm
```

to read a `floatm` value.

Expression `expr` has type `bitsn`, where `n` is the particular architecture's natural pointer size. Its value is the address of the memory location to read from. It will typically contain one or more labels. The size `m` indicates how many bytes to read from that location.

The following example expression reads a 4-byte `bits` from the second byte pointed to by label `p`:

```
bits32[p+1]
```

The address yielded by `expr` is assumed aligned to an `m`-byte boundary. A memory read can optionally be qualified with an alignment flag `{aligna}`. The syntax is:

```
bitsm{aligna}[expr]
floatm{aligna}[expr]
```

A few examples of memory reads with flagged alignment:

- `float64{align4}[label]` reads a 8-byte `float` but assumes that `label` is aligned to a 4 byte boundary.
- `bits32{align1}[label]` reads a 4-byte `bits` but assumes that `label` is aligned to a byte boundary (pointer to a byte).
- `bits8{align4}[label]` reads 1-byte but assumes that `label` is aligned to a 4 byte boundary.

6.3 Operators, precedence, and evaluation order

C++ operators are typed, i.e. there is a different set of operators for the two types provided by C++. The following table lists the available operators for signed `bitss` and `floats`. Operators for *unsigned* `bitss` can be obtained appending the `u` flag to the signed `bits` operators.

Each operator in the table is described in more detail in Section 6.7 and Section 6.8.

<i>Operator</i>	<i>type</i>	<i>flags it can take</i>	<i>class</i>	<i>Description</i>
*	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$	t, u, h	Arithmetic	Integer multiplication
*f	$\text{floatn} \times \text{floatn} \rightarrow \text{floatn}$	t, z, n, p	Arithmetic	Floating point multiplication
/	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$	t, u	Arithmetic	Integer division
/f	$\text{floatn} \times \text{floatn} \rightarrow \text{floatn}$	t, z, n, p	Arithmetic	Floating point division
+	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$	t, u	Arithmetic	Integer addition
+f	$\text{floatn} \times \text{floatn} \rightarrow \text{floatn}$	t, z, n, p	Arithmetic	Floating point addition
-	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$	t, u	Arithmetic	Integer subtraction
-f	$\text{floatn} \times \text{floatn} \rightarrow \text{floatn}$	t, z, n, p	Arithmetic	Floating point subtraction
%	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$	t	Arithmetic	Integer modulo
~	$\text{bitsn} \rightarrow \text{bitsn}$		Bitwise	Complement
&	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$		Bitwise	AND
 	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$		Bitwise	OR
^	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$		Bitwise	XOR
<<	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$		Bitwise	Left shift
>>	$\text{bitsn} \times \text{bitsn} \rightarrow \text{bitsn}$	u	Bitwise	Right shift

Operators should have as arguments expressions of the appropriate type, otherwise the result may be unspecified.

The next table lists the C++ operators in decreasing order of precedence. Operators in the same row have the same precedence. The reader can see that C++ operators follow the precedence order and the associativity of their C counterparts.

<i>Operators</i>	<i>Associates</i>
~	right
* *f / /f %	left
+ +f - -f	left
<< >>	left
&	left
^	left
	left

6.4 Primitives

C++ provides a set of primitive operators besides those described above. The general syntax of a primitive is:

prim_name(*expr*₁, ... *expr*_n)

where *expr*_i are expressions and *prim_name* is the primitive's name. A primitive name is not a *name* in the sense of Section 3.8. Primitive names alone are not expressions that stand for the primitive's entry point address, since primitives are not procedures but built-in operators. Primitives can only be used inside expressions using the syntax given above.

There are **bits** primitives and **float** primitives. The following table lists all the C++ primitives. See Sections 6.7 and 6.8 for detailed information on each particular primitive.

	<i>Primitive</i>	<i>Type</i>
<i>bits primitives</i>	<code>abs%</code>	<code>bitsn → bitsn</code>
	<code>neg%</code>	<code>bitsn → bitsn</code>
	<code>sign%</code>	<code>bitsn → bitsn</code>
<i>float primitives</i>	<code>absf%</code>	<code>floatn → floatn</code>
	<code>exponentf%</code>	<code>floatn → bitsn</code>
	<code>fractionf%</code>	<code>floatn → floatn</code>
	<code>fractpartf%</code>	<code>floatn → floatn</code>
	<code>intpartf%</code>	<code>floatn → floatn</code>
	<code>negf%</code>	<code>floatn → floatn</code>
	<code>predf%</code>	<code>floatn → floatn</code>
	<code>roundf%</code>	<code>floatn × bitsn → floatn</code>
	<code>scalef%</code>	<code>floatn × bitsn → floatn</code>
	<code>signf%</code>	<code>floatn → bitsm</code>
	<code>succf%</code>	<code>floatn → floatn</code>
	<code>truncf%</code>	<code>floatn × bitsn → floatn</code>
	<code>ulpf%</code>	<code>floatn → floatn</code>

6.5 Exception handling

Operators may cause system exceptions such as, for example, overflow or divide-by-zero. Operators can keep record of the exception resulted from their application if they are appended with the `t` (trap) flag. The exception kind is recorded in the global *register*³ `Sys.Indicators`, which is a bit vector with a bit for every kind of exception. To find out which exception has occurred, C++ provides some predefined global constants. They are also bit vectors, with the particular bit that encodes the exception set to 1 and all the others set to 0.

System exceptions and constants are listed in the following table:

<i>Exceptions</i>	<i>lialexcept</i>	→	<code>Sys.IntegerOverflow</code>	
			<code>Sys.FloatingOverflow</code>	
			<code>Sys.Underflow</code>	
			<code>Sys.Undefined</code>	
			<code>Sys.Inexact</code>	<i>IEC 559</i>
			<code>Sys.DivideByZero</code>	<i>IEC 559</i>
			<code>Sys.Invalid</code>	<i>IEC 559</i>
<i>Constants</i>	<i>liainfo</i>	→	<code>Sys.bitsn.MaxSigned</code>	
			<code>Sys.bitsn.MinSigned</code>	
			<code>Sys.bitsn.MaxUnsigned</code>	
			<code>Sys.bitsn.MinUnsigned</code>	
			<code>Sys.floatn.Radix</code>	
			<code>Sys.floatn.Precision</code>	
			<code>Sys.floatn.ExpMin</code>	
			<code>Sys.floatn.ExpMax</code>	
			<code>Sys.floatn.Denorm</code>	
			<code>Sys.floatn.IEC559</code>	
			<code>Sys.floatn.Max</code>	
			<code>Sys.floatn.Min</code>	
			<code>Sys.floatn.MinN</code>	
			<code>Sys.floatn.Epsilon</code>	

³`Sys.Indicators` may be viewed as a global *variable*, but indeed, it is the only possible global variable in a C++ program.

It is easy to find out the kind of exception that has resulted from an operator application using the bitwise operators on `Sys.Indicators` and the appropriate global constants. For example, to capture, handle, and clean up an overflow exception that resulted from the application of a `bits` addition operator, one could write:

```
foo(bits32 y) {
    bits32 x;

    x = y +t y
    if Sys.Indicators & Sys.IntegerOverflow {
        /* Write here code to handle exception */

        /* Clear handled exception */
        Sys.Indicators = Sys.Indicators & ~Sys.IntegerOverflow;
    }
}
```

The `Sys.Indicators` register can be treated as any other register, i.e. it can be cleared, bits can be flipped and so on.

6.6 Casting

6.6.1 `bitsn`

Type: `bitsm` $\rightarrow$ `bitsn`

Accepts flags: `u`

With `t` flag it sets: `N/A`

Description:

If $n > m$, it typecasts returning the higher order bits.

If $n < m$, it typecasts returning a value in which the new higher order bits are either filled with the highest order bits of its argument value (sign extension), or filled with zeroes used with the `u` flag.

Example expression: `bits32('A')` —sign extension. `bits16u(abyte)` —zero fill.

6.6.2 `floatn`

Type: `bitsm` $\rightarrow$ `floatn`

Accepts flags: `t`

With `t` flag it sets: `Sys.FloatingOverflow`

Description:

Typecasts an integer into a floating point number, if `t` is used it raises `Sys.FloatingOverflow` if the argument is too big to fit.

Example expression: `float64(foo)`

6.7 `bits` operators and primitives

All `bitsn` types are bounded and the min and max values are provided as constants: `Sys.bitsn.MaxSigned`, `Sys.bitsn.MinSigned`, `Sys.bitsn.MaxUnsigned` and `Sys.bitsn.MinUnsigned`.

The following definitions will be used to explain the individual operators. I denotes the set of possible integer values. N denotes the subset of I of positive integer values.

$$\begin{aligned}
I &= \{x \in Z \mid \text{Sys.bitsn.MinSigned} \leq x \leq \text{Sys.bitsn.MaxSigned}\} \\
N &= \{x \in Z \mid \text{Sys.bitsn.MinUnSigned} \leq x \leq \text{Sys.bitsn.MaxUnSigned}\} \\
\\
\text{wrap}_I(x) &= x \text{ modulo } (\text{Sys.bitsn.MaxSigned} - \text{Sys.bitsn.MinSigned} + 1) \\
\text{wrap}_I(x) &\in I \\
\\
\text{wrap}_N(x) &= x \text{ modulo } (\text{Sys.bitsn.MaxUnSigned} - \text{Sys.bitsn.MinUnSigned} + 1) \\
\text{wrap}_N(x) &\in N
\end{aligned}$$

6.7.1 + and -

Type: `bitsn × bitsn → bitsn`

Accepts flags: `t` and `u`

With `t` flag it sets: `Sys.IntegerOverflow`

Description:

$$\begin{aligned}
x \pm y &= x \pm y & \text{if } x \pm y \in I \\
&= \text{wrap}_I(x \pm y) & \text{if } x \pm y \notin I
\end{aligned}$$

$$\begin{aligned}
x \pm_t y &= x \pm y & \text{if } x \pm y \in I \\
&= \text{Sys.IntegerOverflow} & \text{if } x \pm y \notin I
\end{aligned}$$

$$\begin{aligned}
x \pm_u y &= x \pm y & \text{if } x \pm y \in N \\
&= \text{wrap}_N(x \pm y) & \text{if } x \pm y \notin N
\end{aligned}$$

$$\begin{aligned}
x \pm_{ut} y &= x \pm y & \text{if } x \pm y \in N \\
&= \text{Sys.IntegerOverflow} & \text{if } x \pm y \notin N
\end{aligned}$$

Example expression: `foo + 17`

6.7.2 *

Type: `bitsn × bitsn → bitsn`

Accepts flags: `t`, `u`, and `h`

With `t` flag it sets: `Sys.IntegerOverflow`

Description:

$$\begin{aligned}
x * y &= x * y & \text{if } x * y \in I \\
&= \text{wrap}_I(x * y) & \text{if } x * y \notin I
\end{aligned}$$

$$\begin{aligned}
x *_t y &= x * y & \text{if } x * y \in I \\
&= \text{Sys.IntegerOverflow} & \text{if } x * y \notin I
\end{aligned}$$

$$\begin{aligned}
x *_u y &= x * y & \text{if } x * y \in N \\
&= \text{wrap}_N(x * y) & \text{if } x * y \notin N
\end{aligned}$$

$$\begin{aligned}
x *_{ut} y &= x * y & \text{if } x * y \in N \\
&= \text{Sys.IntegerOverflow} & \text{if } x * y \notin N
\end{aligned}$$

$$\begin{aligned}
x *_h y &= \text{high}(x * y) & \text{the higher order bits} \\
x *_{uh} y &= \text{high}(x * y) & \text{the higher order bits}
\end{aligned}$$

Example expression: `foo * 17`

6.7.3 /

Type: `bitsn × bitsn → bitsn`

Accepts flags: `t` and `u`

With `t` flag it sets: `Sys.IntegerOverflow` or `Sys.Undefined`

Description:

$$\begin{aligned} x/y &= \lfloor x/y \rfloor & \text{if } \lfloor x/y \rfloor \in I \\ &= \text{wrap}_I(\lfloor x/y \rfloor) & \text{if } \lfloor x/y \rfloor \notin I \\ &= \text{undefined value} & \text{if } y = 0 \end{aligned}$$

$$\begin{aligned} x/ty &= \lfloor x/y \rfloor & \text{if } y \neq 0 \text{ and } \lfloor x/y \rfloor \in I \\ &= \text{Sys.IntegerOverflow} & \text{if } y \neq 0 \text{ and } \lfloor x/y \rfloor \notin I \\ &= \text{Sys.Undefined} & \text{if } y = 0 \end{aligned}$$

$$\begin{aligned} x/uy &= \lfloor x/y \rfloor & \text{if } \lfloor x/y \rfloor \in N \\ &= \text{wrap}_N(\lfloor x/y \rfloor) & \text{if } \lfloor x/y \rfloor \notin N \\ &= \text{undefined value} & \text{if } y = 0 \end{aligned}$$

$$\begin{aligned} x/uty &= \lfloor x/y \rfloor & \text{if } y \neq 0 \text{ and } \lfloor x/y \rfloor \in N \\ &= \text{Sys.IntegerOverflow} & \text{if } y \neq 0 \text{ and } \lfloor x/y \rfloor \notin N \\ &= \text{Sys.Undefined} & \text{if } y = 0 \end{aligned}$$

Example expression: `foo / 17`

6.7.4 %

Type: `bitsn × bitsn → bitsn`

Accepts flags: `t`

With `t` flag it sets: `Sys.Undefined`

Description:

$$\begin{aligned} x\%y &= x - (\lfloor x/y \rfloor * y) & \text{if } y \neq 0 \\ &= \text{undefined value} & \text{if } y = 0 \end{aligned}$$

$$\begin{aligned} x\%ty &= x - (\lfloor x/y \rfloor * y) & \text{if } y \neq 0 \\ &= \text{Sys.Undefined} & \text{if } y = 0 \end{aligned}$$

Example expression: `foo % 17`

6.7.5 neg% and abs%

Type: `bitsn → bitsn`

Accepts flags: `t`

With `t` flag it sets: `Sys.IntegerOverflow`

Description:

$$\text{neg} = \text{wrap}_I(-x) \quad \text{if } -x \notin I$$

$$\text{neg} = \text{Sys.IntegerOverflow} \quad \text{if } -x \notin I$$

$$\text{abs} = \text{wrap}_I(|x|) \quad \text{if } |x| \notin I$$

$$\text{abs} = \text{Sys.IntegerOverflow} \quad \text{if } |x| \notin I$$

Example expression: `neg%(foo)`

6.7.6 sign%

Type: $\text{bits}n \rightarrow \text{bits}n$

Accepts flags: N/A

With **t** flag it sets: N/A

Description:

$$\begin{aligned} \text{sign} &= 0 && \text{if } x = 0 \\ &= -1 && \text{if } x \leq 0 \end{aligned}$$

Example expression: `sign%(foo)`

6.7.7 &, |, and ^

Type: $\text{bits}n \times \text{bits}n \rightarrow \text{bits}n$

Accepts flags: N/A

With **t** flag it sets: N/A

Description:

$$x \& y = x \text{ AND } y \quad \text{Bitwise AND}$$

$$x | y = x \text{ OR } y \quad \text{Bitwise OR}$$

$$x \wedge y = x \text{ XOR } y \quad \text{Bitwise XOR}$$

Example expression: `foo & 17`

6.7.8 ~

Type: $\text{bits}n \rightarrow \text{bits}n$

Accepts flags: N/A

With **t** flag it sets: N/A

Description:

$$\sim x = \text{NOT } x \quad \text{Bitwise complement}$$

Example expression: `~ 17`

6.7.9 << and >>

Type: $\text{bits}n \times \text{bits}n \rightarrow \text{bits}n$

Accepts flags: **u**

With **t** flag it sets: N/A

Description:

$$x \ll n \quad \text{Left shift } n \text{ bits logically}$$

$$x \gg n \quad \text{Right shift } n \text{ bits logically}$$

$$x \gg_u n \quad \text{Right shift } n \text{ bits arithmetically}$$

Example expression: `foo << 17`

6.8 float operators and primitives

The individual operators will just have a short description, for a more through discussion on the different operators consult, LIA-1 or IEC559 as appropriate.

The representation used for floating point numbers is: it is either zero or

$$X = \pm g * r^e = \pm 0.f_1 f_2 \dots f_p * r^e$$

where $0.f_1 f_2 \dots f_p$ is the p -digit *fraction* g (represented in base, or radix, r) and e is the exponent.

The exponent e is an integer in $[emin, emax]$. The fraction digits are integers in $[0, r - 1]$. If the floating point number is *normalized*, f_1 is not zero, and hence the minimum value of the fraction g is $1/r$ and the maximum value is $1 - r^{-p}$.

This description gives rise to five parameters that completely characterize the values of a floating point type and they are available as **bits** valued constants in C++ :

<i>ParameterName</i>	<i>Specifies</i>
<code>Sys.floatn.Radix</code>	<i>base (r)</i>
<code>Sys.floatn.Precision</code>	<i>number of radix digits provided (p)</i>
<code>Sys.floatn.ExpMin</code>	<i>smallest exponent value ($emin$)</i>
<code>Sys.floatn.ExpMax</code>	<i>largest exponent value ($emax$)</i>
<code>Sys.floatn.Denorm</code>	<i>1 if type has denormalized values, 0 if not</i>
<code>Sys.floatn.IEC559</code>	<i>1 if type conforms to IEC559, 0 if not</i>

If `Sys.floatn.IEC559` is equal to 1, most floating point operators support the four different rounding modes defined by IEC559. This is accomplished by adding a flag signifying which rounding mode is desired. The different flags and their rounding modes are:

<i>Flag</i>	<i>Round to</i>
	<i>Nearest</i>
z	<i>Zero</i>
p	<i>Positive Infinity</i>
n	<i>Negative Infinity</i>

A few definitions that we will use for explaining the individual operators:

$$\begin{aligned}
r &\in \mathbb{Z} && \text{The radix of } F \\
p &\in \mathbb{Z} && \text{The precision of } F \\
emin &\in \mathbb{Z} && \text{The smallest exponent of } F \\
emax &\in \mathbb{Z} && \text{The largest exponent of } F \\
denorm &\in \text{Boolean} && \text{Whether } F \text{ contains denormalized values} \\
F_N &= \{0, \pm i * r^{e-p} \mid i, e \in \mathbb{Z}, r^{p-1} \leq i \leq r^p - 1, emin \leq e \leq emax\} \\
F_D &= \{\pm i * r^{e-p} \mid i, e \in \mathbb{Z}, 1 \leq i \leq r^{p-1} - 1, e = emin\} \\
F &= F_N \cup F_D && \text{if } denorm = \text{True} \\
F &= F_N && \text{if } denorm = \text{False} \\
fmax &= \max\{z \in F \mid z > 0\} && = (1 - r^{-p}) * r^{emax} \\
fmin_N &\in \min\{z \in F_N \mid z > 0\} && = r^{emin-1} \\
fmin_D &\in \min\{z \in F_D \mid z > 0\} && = r^{emin-p} \\
fmin &\in \min\{z \in F \mid z > 0\} && = fmin_D \quad \text{if } denorm = \text{True} \\
&&& = fmin_N \quad \text{if } denorm = \text{False} \\
epsilon &= r^{1-p}
\end{aligned}$$

6.8.1 +f, -f, and *f

Type: `floatn × floatn → floatn`

Accepts flags: **t**, **z**, **n**, and **p**

With **t** flag it sets: `Sys.FloatingOverflow` or `Sys.Underflow`

Supported rounding modes (if IEC559): all

Description:

The usual basic arithmetic operators with the proper rounding and trapping notification.

Example expression: `foo +f 17.0`

6.8.2 /f

Type: `floatn × floatn → floatn`

Accepts flags: `t`, `z`, `n`, and `p`

With `t` flag it sets: `Sys.FloatingOverflow` or `Sys.Underflow` or `Sys.Undefined`

Supported rounding modes (if IEC559): all

Description:

The usual basic arithmetic division with the proper rounding and trapping notification.

Example expression: `foo /f 17.0`

6.8.3 signf%

Type: `floatn → bitsm`

Accepts flags: N/A

With `t` flag it sets: N/A

Supported rounding modes (if IEC559): N/A

Description:

$$\begin{aligned} \text{signf}(x) &= 1 && \text{if } x \geq 0.0 \\ &= 0 && \text{if } x = 0.0 \\ &= -1 && \text{if } x \leq 0.0 \end{aligned}$$

Example expression: `signf%(-12.450)`

6.8.4 negf% and absf%

Type: `floatn → floatn`

Accepts flags: N/A

With `t` flag it sets: N/A

Supported rounding modes (if IEC559): N/A

Description:

$$\text{negf}(x) = -x$$

$$\text{absf}(x) = |x|$$

$$\begin{aligned} \text{signf}(x) &= 1.0 && \text{if } x \geq 0.0 \\ &= 0.0 && \text{if } x = 0.0 \\ &= -1.0 && \text{if } x \leq 0.0 \end{aligned}$$

Example expression: `negf%(foo)`

6.8.5 exponentf%

Type: `floatn → bitsn`

Accepts flags: `t`

With `t` flag it sets: `Sys.Undefined`

Supported rounding modes (if IEC559): N/A

Description:

$$\text{exponentf} = \text{Sys.Undefined} \quad \text{if } x = 0.0$$

Example expression: `exponentf%(foo)`

6.8.6 `fractionf%`

Type: `floatn` $\rightarrow$ `floatn`

Accepts flags: N/A

With `t` flag it sets: N/A

Supported rounding modes (if IEC559): N/A

Description:

$$\text{fractionf} = 0 \quad \text{if } x = 0.0$$

Example expression: `fractionf%(foo)`

6.8.7 `scalef%`

Type: `floatn` $\times$ `bitsn` $\rightarrow$ `floatn`

Accepts flags: `t`, `z`, `n`, and `p`

With `t` flag it sets: `Sys.FloatingOverflow` or `Sys.Underflow`

Supported rounding modes (if IEC559): all

Description:

Scales its argument by an integer power of the radix.

Example expression: `scalef%(17.0, 3)`

6.8.8 `succf%` and `predf%`

Type: `floatn` $\rightarrow$ `floatn`

Accepts flags: `t`

With `t` flag it sets: `Sys.FloatingOverflow`

Supported rounding modes (if IEC559): N/A

Description:

$$\text{succf} = \text{Sys.FloatingOverflow} \quad \text{if } x = fmax$$

$$\text{predf} = \text{Sys.FloatingOverflow} \quad \text{if } x = -fmax$$

Example expression: `succf%(17.0)`

6.8.9 `ulpf%`

Type: `floatn` $\rightarrow$ `floatn`

Accepts flags: `t`

With `t` flag it sets: `Sys.Underflow` or `Sys.Undefined`

Supported rounding modes (if IEC559): N/A

Description:

$$\begin{aligned} \text{ulpf} &= \text{Sys.Underflow} \quad \text{if } x \neq 0 \text{ and } r^{e_F(x)-p} \notin F \\ &= \text{Sys.Undefined} \quad \text{if } x = 0 \end{aligned}$$

Example expression: `ulpf%(17.0)`

6.8.10 truncf%

Type: $\text{float}n \times \text{bits}n \rightarrow \text{float}n$

Accepts flags: N/A

With **t** flag it sets: N/A

Supported rounding modes (if IEC559): N/A

Description:

$$\text{truncf} = -\text{truncf}(-x, n) \quad \text{if } x < 0$$

Example expression: `truncf%(17.0, 3)`

6.8.11 roundf%

Type: $\text{float}n \times \text{bits}n \rightarrow \text{float}n$

Accepts flags: **t**, **z**, **n**, and **p**

With **t** flag it sets: `Sys.FloatingOverflow`

Supported rounding modes (if IEC559): all

Description:

$$\text{roundf} = \text{Sys.FloatingOverflow} \quad \text{if } |rn_F(x, n)| > fmax$$

Example expression: `roundf%(17.0, 3)`

6.8.12 intpartf% and fractpartf%

Type: $\text{float}n \rightarrow \text{float}n$

Accepts flags: N/A

With **t** flag it sets: N/A

Supported rounding modes (if IEC559): N/A

Description:

$$\begin{aligned} \text{intpartf} \\ \text{fractpartf} \end{aligned}$$

Example expression: `intpartf%(17.0, 3)`

7 Further Work

This is the TO DO list. It lists all the open issues and the stuff that remains to be added to this manual.

DATE: Mon Apr 20 11:58:56 BST 1998

Expressions and constants

- Add w/ carry operator missing.
- Be more specific about number constants and their syntax. Saying that they're like C's is not enough.
- Regarding pointers.
Pointer arithmetic, e.g.: `float32[ptr + i] = float32[ptr];`
If natural pointer size is 8 then `ptr` has type `word64`; adding integer offset register `i` (say, of type `word32`), if numbers have `word32` type then expression `ptr+i` needs a explicit casting: `ptr + word64(i)`. More architecture-dependancy in the code. (See "Preprocessing")

Preprocessing

C's preprocessing directives can be used in C++ (it would be easy to do, the C++ compiler just passes the C preprocessor to the C++ program before compiling it). It would help for offset-calculation expressions in architectures where the type of an integer is say `word32` and the type of a label is `word64`, and lots of typecast have to be done (since casting is not automatically done), for offsets, in expressions like:

```
word32[label+ word64(3)] = 43;
```

where the integer is typecast, we could have:

```
#ifdef ... /* pointer size == integer size == 8*/
#then
  #define CAST(x) (x)
#else
  /* pointer size != integer size == 4 */
  #define CAST(x) word64((x))

word32[label+CAST(3)] = 43;
```

to avoid to modify the C++ code anytime we want to port the code. For more complicated expressions rewriting will be tedious.

Statements

- Explain better how the optional range improves compiler performance in the switch statement.